Data-Aware Scheduling of Approximate DAG Workflows in Federated Fog Systems

Gayani Gupta*, Amisha Gupta†, Mohsen Amini Salehi*, Antonio Fernández Anta‡,§, Sanjukta Bhowmick*, José Aguilar§,¶
*University of North Texas, USA †UC Berkeley, USA ‡IMDEA Networks Institute, Spain
§IMDEA Software Institute, Spain ¶Universidad de Los Andes, Venezuela
{gayani.gupta, mohsen.aminisalehi, sanjukta.bhowmick}@unt.edu, amisha.gupta@berkeley.edu,
antonio.fernandez@imdea.org, aguilar@ula.ve

Abstract—Flood forecasting, wildfire tracking, and evacuation planning all run as workflows of dependent tasks. During an extreme event, the cloud link often fails, so the work must be done on small fog computers near the disaster area. A single fog site rarely has enough capacity to finish in time, but nearby sites can still reach each other through fast links, so a federation of them is the natural place to run the workflow. Scheduling on a federation is difficult for two reasons. First, a task's input data can take as long to arrive as the task takes to run. Second, many tasks can be run approximately: they give a less accurate result in less time, which is often the right trade-off under a deadline.

We write this scheduling problem as a mixed-integer linear program (MILP). The model decides which fog node runs each task and how much of it runs, so that the workflow's average accuracy is maximized and its deadline is met. The model takes the data size on each edge and the bandwidth of each link as fixed inputs, and decides only where each task runs and how much of it runs; to our knowledge, no earlier model treats these inputs and decisions together. Since the MILP is only practical at moderate sizes, we also build three faster schedulers on the same cost model: linear-program (LP) relaxation with randomized rounding, a bandwidth-aware greedy rule, and a genetic algorithm. We compare all four in simulation, varying deadline slack, federation size, workflow size, link bandwidth, and failure rate. Under tight deadlines, the MILP wins by a clear margin; with generous slack, the MILP and the greedy both reach the accuracy bound while the two stochastic methods stay a few percent below it; and at the largest sizes, the simple greedy is nearly as good while running far faster.

Index Terms—Data-aware scheduling, federated fog systems, directed acyclic graphs, approximate computing, mixed-integer linear programming, genetic algorithms.

I. Introduction

Industrial and emergency applications increasingly run on fog computers: small compute sites close to the sensors, deployed where the data is produced. This is especially important in remote or extreme environments like offshore platforms, deep-sea facilities, or disaster zones, where the link to a cloud data center is slow, intermittent, or simply absent [1], [2]. When the cloud cannot be reached, a local fog site must handle time-critical work on its own, and a single site often does not have enough capacity to finish a workflow before its deadline.

A practical way to add capacity is to federate nearby fog sites. For example, oil rigs a few kilometers apart can maintain fast and stable links to each other even when the wide-area network is down. A federated fog then works like a small local cloud: tasks can be placed on any node, and nodes share data over the inter-fog links [3], [4].

Federation, however, makes scheduling harder, not easier. Real-time industrial applications are usually written as directed acyclic graphs (DAGs), where an edge means that one task must receive another task's output before it can start. Those edges carry real data, such as sensor streams, simulation checkpoints, or model features. Moving that data between nodes takes time that depends on the link bandwidth. Most existing fog schedulers reason only about compute: they ask where a task runs fastest, but not how long it takes to get the task's input there. In a disaster-response workflow, this omission is costly because a "fast" remote placement can lose more time to data transfer than it saves in execution.

There is a second factor that data-unaware schedulers also ignore: tasks can run approximately. Many analytics tasks produce a less accurate result in less time: sampling a simulation fewer times, truncating a feature pipeline, or stopping an iterative method early. When a deadline is tight, running a task at 80% accuracy may be the difference between a warning that arrives in time and one that arrives too late.

This paper integrates both aspects into one scheduling model. The contributions are:

- A data- and bandwidth-aware formulation of approximate DAG scheduling in federated fog systems as a mixed-integer linear program (MILP, Section III). The objective is to maximize the average accuracy of the workflow; the constraints are precedence, communication delay, placement eligibility, and a single workflow deadline. To our knowledge, the per-edge data size, the per-link bandwidth, and the per-task execution fraction have not previously been modeled jointly in this setting.
- Three scalable schedulers that share the same cost model as the MILP: LP relaxation with randomized rounding, a bandwidth/data-aware greedy algorithm,

and a genetic algorithm (Section IV).
- A simulation study that varies the deadline slack, the fog-network size, the workflow size, the task failure rate, and the link bandwidth, and reports where each method wins (Section V).
- A correctness argument and a linearization of the model suitable for standard MILP solvers (the Appendix).

An earlier version of the scheduling model appeared in a conference paper [5]. The present work extends it by stating the complete data- and bandwidth-aware formulation with its proofs and linearization, by adding the failure-rate experiments, and by positioning the model against the DAG-scheduling literature (Section II).

Section II reviews related work, Section III defines the model and problem, Section IV describes the scalable schedulers, and Section V reports the experiments.

II. Related Work

A. DAG Scheduling on Heterogeneous Resources

Scheduling a DAG of tasks on heterogeneous processors is a well-studied problem. The standard reference heuristic is HEFT (Heterogeneous Earliest Finish Time), introduced by Topcuoglu, Hariri, and Wu [6]. HEFT first computes for each task an upward rank, roughly the remaining work plus communication on the longest path to the end of the graph, and then places each task in rank order on the processor where it finishes earliest. Our model uses the same convention: the transfer cost lives on the DAG edge. The main difference is what a task is allowed to be. HEFT assumes that a task runs completely or not at all, and the schedulers built on it keep that assumption. So do the DAG schedulers used in mobile edge computing [7] and the learned schedulers that use a graph neural network with reinforcement learning [8]. In our model, a task can run partially. The execution fraction l_{ia} is a decision variable, so if the workflow cannot finish on time at full quality, it can still finish with less accuracy.

Workflow engines work at a different level. Tools like Pegasus [9], [10], Snakemake [11], Nextflow [12], StreamFlow [13], and DagOnStar [14], [15] take a workflow, resolve its dependencies, and move the data between machines. They decide how a placement is carried out, but the placement decision itself is usually left to simple matchmaking or to a site scheduler. Our work fills that gap: it is the optimization layer that decides where each task should run, with the data transfer cost part of the decision. Other work has studied how to partition a task graph for data-intensive workflows [16], but it does not model per-link bandwidth or partial execution.

B. Fog Scheduling and Federation

There are several good surveys on resource allocation in fog and edge systems [17], [18], [4], [19]. A number of papers propose federating nearby fog sites so that they can share work in industrial settings [3], stream processing [20], and emergency response [21]; the fast short-range links this relies on are studied in [22], including offshore and marine settings [23]. Inside a federation, researchers have optimized how to partition and place application components for throughput [24] and load balancing [25], designed offloading policies to meet quality-of-service targets [26], and studied energy use [27]. Learning-based schedulers adapt placements as the workload changes [28], [29]; [30] surveys the broader edge-offloading landscape.

These approaches share one limitation that matters for our problem: they are compute-centric. They model where a task runs and how fast it runs, but the data that has to move between dependent tasks enters the model weakly or not at all. In a federated fog, the links between nodes are the scarce resource, so the transfer cost cannot be ignored. Our formulation includes the data volume on every edge and the bandwidth on every link directly in the optimization, together with the option to run tasks partially.

C. Robustness and Approximate Execution

Trading result quality for resources is not a new idea in scheduling. Probabilistic task pruning drops low-value tasks, so the schedule stays feasible under load [31]. Stochastic optimization has been used to make resource allocations robust to uncertain runtimes [32]. We use a softer version of the same idea: instead of dropping a task, the scheduler can run part of it, and the goal is to keep the total delivered accuracy as high as possible. This difference matters in disaster response, where dropping the alerting task is not an option, but running a faster, slightly less accurate version of it may be.

III. System Model and Problem Statement

A. Setting

We consider a federation of fog nodes $F = \{f_1, \dots, f_m\}$ deployed in a disaster-prone area such as a cluster of offshore oil platforms (Fig. 1). Each pair of nodes is connected by a link of known bandwidth, and the federation operates even when the cloud is unreachable. A workflow arrives as a DAG $G = (T, E)$, where each node $T_i \in T$ is a computational task, and each edge $(T_j, T_i) \in E$ is both a precedence constraint and a data transfer of size D_{ji}.

If task T_j runs on node f_b and its successor T_i runs on node f_a, the input of T_i has to cross the link (f_b, f_a) with bandwidth B_{ba}. That transfer takes:

$$c_{ji}^{ba} = \begin{cases} 0, & f_b = f_a, \\ \dfrac{D_{ji}}{B_{ba}}, & f_b \neq f_a. \end{cases} \tag{1}$$

This is the central cost in the model: placing a task is not only about how fast a node computes, but also about how long its input data take to arrive.

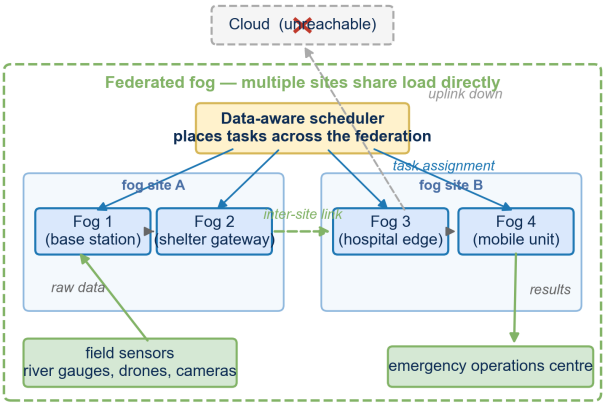


Fig. 1: Federated fog deployment for disaster response. Sensors feed a workflow of dependent tasks; the scheduler assigns each task to one fog node, priced by both compute time and inter-node data-transfer cost.

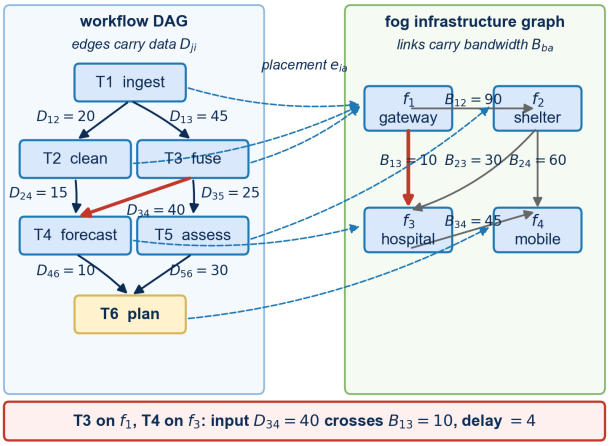


Fig. 2: Example disaster-response workflow. Edge labels are data sizes D_{ji}; dashed arrows show an assignment e_{ia} of tasks to federated fog nodes. Running a task off the node that produced its input pays the transfer delay of Eq. (1).

B. Assumptions

- Tasks are non-preemptive: once started on a node, a task finishes there.
- Each task runs on at most one node.
- A task can receive input from all of its predecessors simultaneously when the links allow it.
- Concurrent tasks on the same node do not slow each other down. (Each node is assumed to offer unbounded parallelism.)
- A task can be executed approximately: executing fraction l of a task takes fraction l of its full execution time and yields fraction l of its accuracy.

The last assumption deserves a closer look. Running T_i at l_{ia}=0.5 produces half of a full-quality result: an iterative solver stopped after half its iterations, a video pipeline processing every other frame, a training run cut at half its epochs. Time and accuracy are taken to grow linearly in the fraction run — time because the same work is simply executed fewer times, accuracy because for incremental algorithms (iterative refinement, Monte Carlo estimation, sampled processing) quality grows roughly with the effort spent. This is a first-order model, not a law: some tasks are all-or-nothing, accuracy curves often saturate rather than rise uniformly, and the fixed cost of starting a task is ignored. We keep it because it makes the deadline–accuracy trade-off explicit and keeps (2)–(10) linear.

TABLE I: Parameters and decision variables.

Symbol	Meaning
T_i	task i of the workflow DAG
f_a	fog node a
t_{ia}	full execution time of T_i on f_a
A_i	accuracy delivered if T_i runs completely
D_{ji}	data transferred from T_j to T_i
B_{ba}	bandwidth of link (f_b, f_a)
c_{ji}^{ba}	communication delay, Eq. (1)
P_{ia}	1 if T_i may run on f_a, else 0
in_i	predecessors of T_i in the DAG
$t_{\max}$	workflow deadline
e_{ia}	1 if T_i is assigned to f_a
l_{ia}	fraction of T_i executed on f_a
M_{ia}	earliest time T_i can start on f_a
y_i	completion time of T_i

C. The Scheduling Problem

Table I lists the notation. Each task T_i has a full-accuracy value A_i, an execution time t_{ia} on each node f_a, and a placement-eligibility indicator P_{ia}. The workflow must finish by a deadline $t_{\max}$.

The problem is then to choose, for every task, which node runs it and how much of it runs, so that the average delivered accuracy is as high as possible and that every task finishes by $t_{\max}$.

D. MILP Formulation

The objective is to maximize the average accuracy over the workflow:

$$\max \quad \frac{1}{n} \sum_{T_i \in T} A_i \sum_{f_a \in F} l_{ia}. \tag{2}$$

subject to

$$\sum_{f_a \in F} e_{ia} = 1, \qquad \forall T_i \in T, \tag{3}$$

$$e_{ia} \le P_{ia}, \qquad \forall T_i,\ f_a, \tag{4}$$

$$0 \le l_{ia} \le e_{ia}, \qquad \forall T_i,\ f_a, \tag{5}$$

$$M_{ia} \ge y_j + \sum_{f_b \in F} e_{jb}\, c_{ji}^{ba}, \qquad \forall f_a,\ \forall T_i,\ \forall T_j \in \mathrm{in}_i, \tag{6}$$

$$M_{ia} = 0, \qquad \forall f_a,\ T_i : \mathrm{in}_i = \varnothing, \tag{7}$$

$$y_i = \sum_{f_a \in F} e_{ia}\big(M_{ia} + t_{ia} l_{ia}\big), \qquad \forall T_i, \tag{8}$$

$$y_i \le t_{\max}, \qquad \forall T_i, \tag{9}$$

$$e_{ia} \in \{0, 1\}. \tag{10}$$

Constraints (3)–(4) place every task on exactly one permitted node. Constraint (5) ties the execution fraction to the assignment: a task can only be (partially) run where it is placed, and at most one l_{ia} per task is non-zero. The case $l_{ia} = 0$ means the task is placed but never runs. A skipped task earns no accuracy, but it still hands its output data D_{ji} to every successor, so the transfer cost in (1) is charged whether or not the task ran. Skipping is rare in practice (Fig. 8); every certified-optimal run executes all tasks at least partially. The constraint (6) is the data-aware term: a task cannot start before every predecessor has finished and its output has crossed the link, which is where D_{ji} and B_{ba} enter the optimization through Eq. (1). Equation (8) defines the completion time and (9) enforces the deadline. The products $e_{ia}M_{ia}$ in (8) make this a mixed-integer nonlinear constraint as written; the Appendix gives the standard linearization (auxiliary variables $z_{ia} = e_{ia}M_{ia}$) together with the correctness argument.

E. Complexity

Scheduling problems of this kind are NP-hard already on unrelated parallel machines [33], and precedence, communication, and approximation make them only harder. We therefore treat the MILP as the benchmark: it gives the best achievable accuracy on instances small enough to solve exactly, and it defines the constraints that every scalable method must respect. For larger workflows, we use three cheaper methods, described next: randomized rounding [34], a greedy scheduler, and a genetic algorithm.

IV. Scalable Scheduling Algorithms

All three methods take the same inputs as the MILP: the DAG, the times t_{ia}, the data sizes D_{ji}, the bandwidths B_{ba}, the eligibility flags P_{ia}, and the deadline $t_{\max}$. Because they all evaluate the communication term (1), they are all bandwidth-aware.

A. LP Relaxation with Randomized Rounding

The first method relaxes the binary variables of Section III-D to $e_{ia} \in [0, 1]$ and solves the resulting LP—namely, the linearized formulation of the Appendix, since (8) contains the product $e_{ia}M_{ia}$ and is only linear after the z_{ia} substitution. Let r_{ia} be the value the LP gives to e_{ia}; we can read it roughly as the probability that node f_a is a good place for T_i. We then assign each task to a node by sampling this distribution, renormalized over the eligible nodes $\{a : P_{ia} = 1\}$. Once the assignment is fixed, a second LP solve sets the execution fractions l_{ia} and the timing values.

The rounding inherits bandwidth awareness from the LP itself: a placement that needs a large D_{ji} over a slow link gets a small r_{ia} in the relaxation and is therefore rarely sampled. Assignments that turn out infeasible are discarded and re-sampled. The method sits between the MILP and the heuristics: it has no optimality guarantee, but its sampling distribution comes from an actual optimization rather than a rule of thumb.

B. Bandwidth- and Data-Aware Greedy Scheduler

The greedy scheduler processes the DAG in dependency order and assigns each ready task to the node that finishes it earliest. Among the ready tasks, it picks the one with the highest priority, which mixes accuracy with urgency:

$$\pi_i \;=\; \beta \cdot A_i \;+\; (1 - \beta) \cdot \mathrm{CP}_i, \tag{11}$$

where CP_i is the critical-path length from T_i to a sink under the current t_{ia} and c_{ji}^{ba}, and $\beta \in [0, 1]$ trades accuracy importance against schedule urgency (default $\beta = 0.5$; ties break by accuracy density, i.e. A_i divided by the task's mean eligible execution time). Since A_i and CP_i have different units, (11) is a heuristic priority score rather than a physical quantity: A_i is an order-one accuracy weight while CP_i is measured in time units, so β controls the trade-off at the relative scale of the two ranges rather than in absolute terms.

The scheduler keeps a single global execution fraction $\ell \in [\ell_{\min}, 1]$, applied to every task, and decreases it by a step δ whenever the produced schedule misses the deadline, down to a floor $\ell_{\min} > 0$ at which the run is declared infeasible:

Complexity: Each task scans all m nodes and, for each candidate, takes a maximum over its in-neighbours, so one pass costs $O(nm + m|E|)$; the number of outer passes is bounded by $\lceil (1 - \ell_{\min})/\delta \rceil$. The floor matters when communication alone already exceeds $t_{\max}$—in the 0.05-bandwidth regime of Section V-F, for instance, some placements cannot meet the deadline at any execution fraction—and the loop exits there instead of decreasing ℓ without bound.

The greedy is a practical method rather than just a baseline: it accounts for the deadline through ℓ, for bandwidth through c_{ji}^{ba}, and for node speed through t_{ia}. What it cannot do is look ahead: a cheap early decision can force an expensive one later. The genetic algorithm addresses exactly that.

Algorithm 1 Bandwidth/Data-Aware Greedy DAG Scheduler

$\ell \leftarrow 1$; compute priorities π_i by (11)
repeat
  $y_i \leftarrow \infty$ for all T_i; ready set $\mathcal{R} \leftarrow$ source tasks
  while $\mathcal{R} \neq \varnothing$ do
    pick $T_i \in \mathcal{R}$ with maximum π_i
    $y^\star \leftarrow \infty$; $a^\star \leftarrow \perp$
    for each $f_a \in F$ with $P_{ia} = 1$ do
      $M_{ia} \leftarrow \max_{T_j \in \mathrm{in}_i} \big(y_j + \sum_{f_b \in F} e_{jb}\, c_{ji}^{ba}\big)$
      $y_i^{(a)} \leftarrow M_{ia} + t_{ia}\,\ell$
      if $y_i^{(a)} < y^\star$ then
        $y^\star \leftarrow y_i^{(a)}$; $a^\star \leftarrow a$
      end if
    end for
    $e_{ia^\star} \leftarrow 1$; $y_i \leftarrow y^\star$; release newly-ready successors into $\mathcal{R}$
  end while
  $Y_{\max} \leftarrow \max_i y_i$
  if $Y_{\max} > t_{\max}$ then
    $\ell \leftarrow \ell - \delta$ {e.g. $\delta = 0.005$}
  end if
until $Y_{\max} \le t_{\max}$ or $\ell < \ell_{\min}$
if $Y_{\max} > t_{\max}$ then
  return infeasible {even $\ell = \ell_{\min}$ misses the deadline}
end if

C. Genetic-Algorithm Scheduler

A chromosome encodes a full placement plus a per-task execution fraction: an assignment vector $\mathbf{f}$ with $f(i) \in \{a : P_{ia} = 1\}$, and a fraction vector $\boldsymbol{\ell} \in [0,1]^n$ that maps to $l_{i,f(i)}$. Fitness is evaluated by a forward pass over the DAG,

$$M_{i,f(i)} = \max_{T_j \in \mathrm{in}_i} \Big(y_j + c_{ji}^{f(j),f(i)}\Big), \qquad y_i = M_{i,f(i)} + t_{i,f(i)}\ell_i, \tag{12}$$

and the fitness to maximize is

$$\Phi = \frac{1}{n}\sum_i A_i\,\ell_i - \lambda_1\,\mathbb{I}[Y_{\max} > t_{\max}] - \lambda_2\,\frac{(Y_{\max} - t_{\max})_+}{t_{\max}}, \tag{13}$$

where $Y_{\max} = \max_i y_i$, $\mathbb{I}[\cdot]$ is the indicator function, and $\lambda_1 \gg \lambda_2$ penalize deadline violation. Infeasible individuals ($P_{i,f(i)} = 0$) receive $\Phi = -\infty$.

Because the GA evaluates complete assignments instead of committing tasks one at a time, it can find structures the greedy misses; for example, it can group a data-heavy subgraph on one well-connected node. In practice, a few dozen generations with a modest population are enough.

Algorithm 2 Genetic Algorithm for Bandwidth/Data-Aware Fog Scheduling

initialize population with feasible $(\mathbf{f}, \boldsymbol{\ell})$; seed co-location of predecessors joined by large D_{ji}
repeat
  evaluate Φ of each individual via (13)
  select parents (tournament or roulette)
  one-point crossover on $\mathbf{f}$ and $\boldsymbol{\ell}$ independently
  mutate: reassign $f(i)$ within $\{a : P_{ia} = 1\}$; jitter $\ell_i \leftarrow \mathrm{clip}_{[0,1]}(\ell_i + \mathcal{N}(0, \sigma^2))$
  repair: bias toward co-locating T_i with a predecessor T_j when D_{ji} is large and the link is slow
  elitism: keep top-k individuals
until maximum generations reached or no improvement

V. Performance Evaluation

A. Experimental Setup

All experiments use a custom discrete-event simulator. Workloads are synthetic disaster-response DAGs serialized in JSON, each task carrying an execution-time vector t_{ia}, edge data sizes D_{ji}, a full-accuracy value A_i, and eligibility P_{ia}. The fog federation is generated with heterogeneous node speeds and link bandwidths, and communication delays follow Eq. (1). The parameters are chosen so that data transfers are contested rather than negligible or prohibitive: task times $t_{ia} \in [1, 5]$ (normalized units), edge data sizes $D_{ji} \in [5, 50]$, and link bandwidths $B_{ba} \in [10, 100]$, so that D_{ji}/B_{ba} is comparable to task execution time; node speeds vary within $[0.5, 2]\times$ and roughly 15% of task–node pairs are ineligible, modeling missing sensor feeds or software dependencies.

- Test datasets: four schedulable instance sets spanning the sizes used throughout: 10 tasks/3 nodes, 100 tasks/5 nodes, 500 tasks/8 nodes, and 1000 tasks/10 nodes. Each is generated with the parameter ranges above and a fixed random seed per configuration, so every reported number is reproducible. Larger workflow structures (DAG edge lists up to 10^6 tasks) and federations up to 10^6 nodes are generated and analyzed in the companion notebook; they exercise graph generation and statistics, not the dense schedulable-instance path, whose $n \times n$ edge-data matrix does not fit at that scale.
- Workflow sizes: small (10 tasks), medium (100 tasks), large (500 tasks), and extra-large (1000 tasks); the two large datasets use a sparser DAG (density 0.02 vs. 0.15) so the edge count stays realistic at that scale.
- Federation size: $N \in \{2, 4, 6, 8, 10\}$ fog nodes.
- Deadline slack: the deadline is set as $t_{\max} = \alpha \cdot T_{\mathrm{cp}}$, where T_{cp} is the critical-path length of the workflow at full execution and $\alpha \in \{1.0, 1.2, 1.3, 1.5, 2.0\}$ is the slack factor.
- Task failures: a per-task failure rate from 0% to 50% under a retry-once model: a failed task is re-executed once on the same node, so it costs twice its execution time but keeps its accuracy if the retry still meets the deadline.
- Link bandwidth: all inter-node bandwidths scaled by

a factor in $\{1.0, 0.5, 0.2, 0.1, 0.05\}$ with the deadline unchanged, modeling link degradation during an event.

- Methods compared: the MILP of Section III-D, randomized rounding, the greedy of Section IV-B, and the GA of Section IV-C. Two reference configurations bracket the range: a tight deadline ($\alpha = 1.0$) and a max-accuracy bound ($t_{\max} \to \infty$) that upper-bounds achievable accuracy. For the bandwidth experiment, we also run a bandwidth-blind variant of the greedy. It is the same algorithm, but it chooses nodes as if transfers were free; because it never sees communication delay, it does not run the deadline-driven ℓ-shrinking loop of Algorithm 1, so it reports the bound-level score even when its schedules later miss the deadline under the real link costs.

Unless a varied parameter says otherwise, every experiment instance uses the default configuration: n=50 tasks, m=5 fog nodes, slack $\alpha = 1.5$, and edge density 0.15, with five replications per point at different seeds. All instances come from the same generator under these fixed seeds, so every reported number is reproducible. In particular, the failure and bandwidth-degradation experiments below use this default $\alpha = 1.5$. The MILP is solved with a 60 s time limit and skipped above 800 assignment variables ($n\,m$); a time-limited run reports its best incumbent, which can sit below a bound that a heuristic happens to reach—the bound is then achievable, but the truncated solve has not certified it. Every MILP run that finished below another method on the same instance was therefore re-solved with a 300 s limit and a 10^{-6} optimality gap; the figures and tables report these certified scores, and no reported MILP score sits below a heuristic score on the same instance.

Metrics recorded per run: the achieved accuracy score $\bar{A} = \frac{1}{n}\sum_i A_i\,\ell_i$, where $\ell_i = \sum_a l_{ia}$ is the fraction of task T_i executed. This is what the MILP maximizes. Because each A_i is a per-task accuracy weight rather than a probability, the score is not bounded by 1 and is read relative to the max-accuracy bound of the same instance, drawn as a dashed line in every figure. Makespan and deadline adherence are also recorded; all values are averaged over repeated random DAGs.

Table II reports the per-dataset test runs; these use fixed seeded instances that differ from the replication instances behind the experiment figures, so their scores and bounds need not coincide with the experiment means. Two trends are already visible: the greedy and the MILP achieve the same score wherever both run, because under the tested slack all tasks execute fully and placement only moves the makespan; and the GA's score drops as the workflow grows, because its fixed population budget must cover an n-dimensional search space. Note that the MILP maximizes accuracy, not makespan: on the medium dataset it and the greedy reach the same score while their makespans differ (60.94 vs. 43.11), since any placement that runs all tasks fully and meets the deadline is optimal for the objective.

TABLE II: Per-dataset test runs on the seeded instances (small: α=1.2; medium, large, xlarge: α=1.5). Score is the average achieved accuracy $\bar{A}$; all listed schedules meet the deadline. "–" means the method was not run on that dataset: the MILP is capped at $n\,m \le 800$ assignment variables, and randomized rounding is skipped on the two largest datasets because it re-solves the LP once per trial (40 trials per run here). Rounding does appear in the workflow-size experiment of Section V-D at 1000 tasks, where its trial count is reduced. The experiment figures use different seeds and instance sizes than these fixed test datasets, so their max-accuracy bounds are not the same values reported here.

Dataset	n	m	Method	Score	Makespan	Time (s)
small	10	3	MILP	1.637	17.98	0.013
			Rounding	1.637	17.98	0.16
			Greedy	1.637	16.84	0.000
			GA	1.637	18.90	0.27
medium	100	5	MILP	1.443	60.94	1.37
			Rounding	1.324	62.17	24.83
			Greedy	1.443	43.11	0.003
			GA	1.274	61.26	5.19
large	500	8	Greedy	1.419	50.15	0.04
			GA	1.173	71.26	21.43
xlarge	1000	10	Greedy	1.406	83.14	0.14
			GA	1.126	119.85	73.01

Reproducibility. The simulator used for the evaluation (instance generation, the four schedulers, and the experiment drivers) is provided with this submission in code/fogsim.py; generated workflow instances are in data/instances/ and per-run results in data/results/, with per-run MILP solver statistics (model size, solver status, wall time, and optimality gap) in revisions/results/task-02/milp_solver_details.csv and per-run counts of skipped tasks ($l_{ia} = 0$) in revisions/results/task-04/skipped_tasks.csv. The implementation depends only on NumPy, SciPy (HiGHS for the MILP/LP solves), and pandas. An executed Jupyter notebook (notebooks/experiments.ipynb) documents the datasets, per-dataset test runs, the workflow and resource graphs, the asymptotic runtime analysis, and the scaling limitations encountered (solver caps, sparse-constraint requirements, and fixed GA budgets). Code and data are publicly available at https://github.com/gayanigupta/data-aware-scheduling [35].

B. Deadline Slack

Fig. 3 shows the reported accuracy score against the slack factor α. The methods separate at the tight deadline: at $\alpha = 1.0$ the MILP stays within about 3% of the bound (1.38 vs. 1.43), rounding follows at 1.25, the greedy at 1.23, and the GA lowest at 1.20, about 16% below the bound. Once the deadline relaxes, the picture changes. From $\alpha = 1.2$ onward, the greedy reaches the bound exactly, and

after the certification re-solves so does the MILP: the runs that were still improving when the 60 s limit hit all certified the bound within 300 s, confirming that the gap was a truncated incumbent rather than a weaker optimum. The two stochastic methods never fully close the gap in our runs: rounding stays about 8% below the bound at $\alpha = 1.2$ and roughly 2.5% below at $\alpha = 1.5$–2.0, while the GA trails by 3–10% across the range. The lesson is twofold: the scheduling choice matters most when the workflow barely fits its deadline, and a fixed sampling budget does not recover the bound even when one exists.

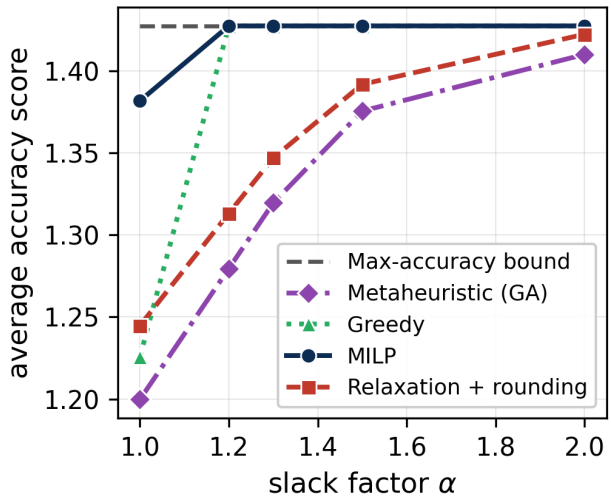


Fig. 3: Reported average-accuracy score vs. deadline slack factor α ($t_{\max} = \alpha \cdot T_{\text{cp}}$).

C. Federation Size

Fig. 4 shows the reported accuracy score as the federation grows from $N = 2$ to $N = 10$ nodes. These instances use the default slack $\alpha = 1.5$, so accuracy is already close to the ceiling and extra nodes do not raise it; what they add is room to schedule. The MILP and the greedy match the bound exactly at every size. The two stochastic methods drift downward as the federation widens: the GA goes from about 1.5% below the bound at $N = 2$ to about 8% below at $N = 10$, and randomized rounding matches the bound only at $N = 2$ before falling about 8% short at $N = 10$. Both have a fixed search or sampling budget, and that budget covers a shrinking fraction of the placement space as N grows. The lesson is that extra federation capacity does not raise accuracy once the deadline is met; its value is in the feasible placements it opens when the deadline is tight.

D. Workflow Size

Fig. 5 shows accuracy for DAGs of 10, 100, and 1000 tasks (the small, medium, and extra-large scales of Table II). These experiment instances are generated per replication with different seeds than the Table II datasets,

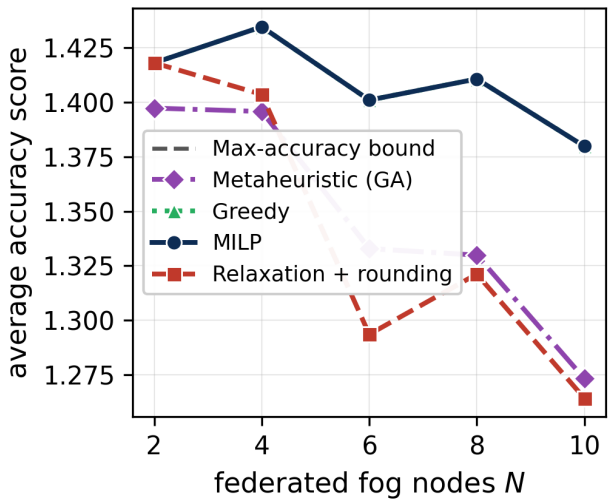


Fig. 4: Reported average-accuracy score vs. number of federated fog nodes N.

so their bounds differ (about 1.41 at 10 tasks here vs. 1.637 on the small test dataset). At 10 tasks all four methods sit on the bound (1.41); at 100 the MILP and the greedy match the bound (1.40), rounding follows at 1.35, and the GA is the first to drift at 1.26. At 1000 tasks, the MILP is no longer reported, since it exceeds our 800-assignment-variable cap. Of the remaining methods, the greedy stays on the bound (1.40), randomized rounding follows at 1.37 (each LP re-solve costs about 93 s here, so its trial count is reduced to four per run), and the GA falls to 1.10. The reason is that the GA's population and generation budget stay fixed while the search space grows with n, so its accuracy declines with size. The greedy, in contrast, makes one dependency-ordered pass, so it scales almost linearly and loses nothing in this regime, where the slack is loose enough to run most tasks fully. The right panel puts a wall-clock price on the same runs: at 1000 tasks the greedy answers in about 0.1 s, while the GA takes roughly 10^2 s and rounding a few hundred seconds more on top of that.

E. Task Failures

We model failures with a retry-once rule: when task T_i fails on its node, it runs once more in place, paying t_{ia} a second time, and it keeps its accuracy if the retry still finishes by the deadline. Fig. 6 shows score and deadline feasibility as the per-task failure rate rises from 0% to 50%. Scores do not move with the failure rate, since retries preserve accuracy; the methods separate on feasibility instead. The greedy is the only method whose schedule stays feasible at every rate: its placements leave enough slack to absorb the retries. The MILP and the GA meet the deadline in about 60% of runs once failures occur, and randomized rounding in about 20%. The curves are nearly flat because at the experiment default of 50 tasks

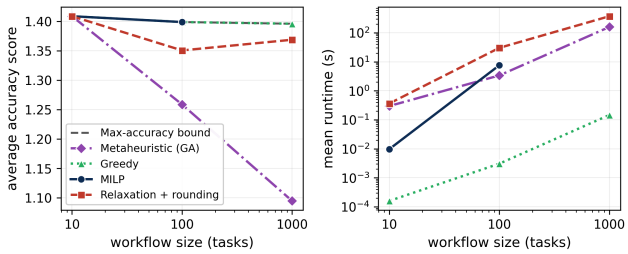


Fig. 5: Reported average-accuracy score (left) and mean runtime (right, log scale) vs. DAG size. The greedy tracks the max-accuracy bound at every size while answering in fractions of a second; randomized rounding stays within a few percent but costs hundreds of seconds per run at 1000 tasks, and the GA degrades in score under its fixed search budget. The MILP is not reported beyond its assignment-variable cap.

even a 10% per-task rate makes at least one failure almost certain, so feasibility is decided by whether the schedule can absorb typical failure sets rather than by the rate itself.

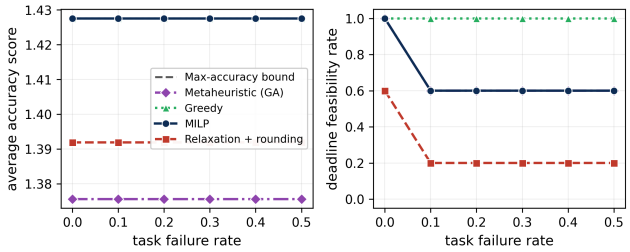


Fig. 6: Accuracy score (left) and deadline feasibility rate (right) vs. task failure rate under the retry-once model. Retries preserve accuracy but consume extra time, so the methods separate on feasibility: schedules with more slack absorb failures longer.

F. Link Bandwidth Degradation

The final experiment stresses the communication cost that motivates the model. On the default instance (n=50, m=5, α=1.5), every inter-node bandwidth B_{ba} is multiplied by a factor from 1.0 down to 0.05 while the deadline stays fixed, which is what a federation faces when links deteriorate during an event. To show what data-aware placement is worth, we add a bandwidth-blind baseline: the same greedy algorithm as Section IV-B, but choosing placements as if all transfers were free, and then evaluated against the real communication delays.

Fig. 7 shows the two metrics that matter. On score alone, the blind baseline looks perfect: it runs every task fully and reports the bound at every degradation level. The right panel shows why that is misleading: the blind schedules start missing the deadline already at half bandwidth (80% feasible at 0.5, 20% at 0.2) and are infeasible in every run at 0.1 and below. The data-aware schedulers behave differently. The MILP stays feasible at every level and loses only about 10% of score at 0.05, shrinking execution fractions to fit the deadline. The greedy also stays feasible down to 0.1, but pays for it with accuracy (score 0.80 at 0.1, and mostly infeasible at 0.05). The GA is mostly feasible down to 0.1 (80% at 0.2, 40% at 0.1) and fails at 0.05. Randomized rounding holds at 0.5, but its sampled assignments largely stop surviving the feasibility LP below that (60% feasible at 0.2, none at 0.1 and lower); this is a limit of our implementation, not a proof that no feasible rounding exists.

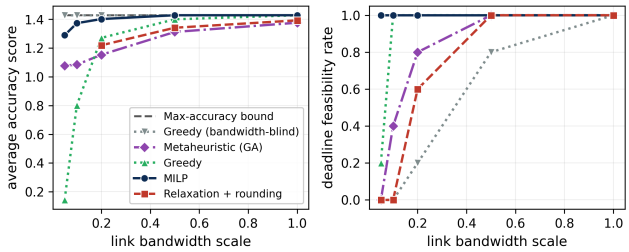


Fig. 7: Accuracy score (left) and deadline feasibility rate (right) as all link bandwidths are scaled down. The bandwidth-blind baseline reports bound-level scores but its schedules miss the deadline whenever links drop below roughly half capacity; the data-aware methods trade accuracy for feasibility instead.

G. Discussion

The experiments point to three conclusions. First, the choice of scheduler matters most when the deadline binds: at $\alpha = 1.0$ the methods separate, with the MILP ahead and the heuristics trailing, while by $\alpha = 1.5$ the MILP and the greedy coincide with the bound and only the two stochastic methods stay a few percent below it. Second, accuracy under a binding deadline is not free in wall time: at 1000 tasks, averaged over the experiment replications, the greedy finishes in about 0.15 s, while randomized

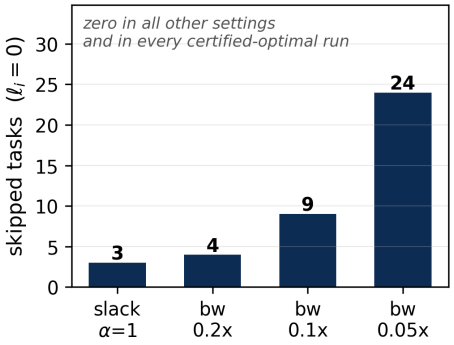


Fig. 8: Skipped tasks (ℓ_i=0) per setting. Skips occur only in runs stopped at the time limit; no certified-optimal solution skips a task.

rounding spends about 93 s per LP re-solve (roughly 372 s per run even with only four trials) and the GA about 159 s for a markedly worse score. (Table II's single seeded xlarge run reports 73 s for the GA; the two figures measure different instances and should not be conflated.) Third, under the retry-once failure model, the schedulers separate on feasibility rather than accuracy because retries preserve score but consume time, so placements that leave slack tolerate more failures.

Bottlenecks and shortcomings. The evaluation surfaced limits that delimit where the formulation applies. (i) MILP scalability: the model has $n \cdot N_{\text{fog}}$ binary assignment variables, and we cap runs at 800 with a 60 s solver time limit below that (300 s for the certification re-solves described in the setup). Of the 120 experiment runs, 18 reached the limit, all at α=1.0 or under degraded bandwidth—exactly the hard regime. Therefore, it cannot certify optimality on large workflows, and even within the cap a truncated run reports an incumbent that can sit below an achievable bound. (ii) LP model size: the relaxation introduces one start-time constraint per (task, predecessor, node) triple, about 10^5 rows at n=1000. Constraints must be assembled sparsely, and each solve costs tens of seconds, a cost rounding pays per trial. (iii) Fixed GA budget: the population and generation count do not grow with n, which is why the GA's score degrades visibly in Fig. 5; size-adaptive budgets or seeded search should close most of that gap. (iv) Failure semantics: the retry-once model differentiates schedulers through feasibility, but richer treatments such as cross-node replication or checkpointing are left out of the model. (v) Dense instances: schedulable instances store per-edge data densely, so very large DAGs, which the companion notebook generates structurally up to 10^6 tasks, cannot yet be carried as schedulable instances; sparse instance storage and decomposed formulations would extend the same cost model to genuinely large deployments.

VI. Conclusion and Future Work

This paper formulated data- and bandwidth-aware scheduling of approximate DAG workflows in federated fog systems as a mixed-integer linear program, and gave three scalable schedulers built on the same cost model. In simulation, when deadlines are tight, the MILP earns a clear accuracy advantage that justifies its solve time; when slack is generous, the MILP and the greedy coincide at the bound while the stochastic methods stay a few percent below it; and at large workflow sizes, the lightweight greedy holds its own while the GA's fixed search budget becomes its weakness. Under the retry-once failure model, the schedulers separate on deadline feasibility rather than accuracy.

Future work has four directions, each addressing a shortcoming documented above. Sparse and decomposed formulations: sparse instance storage and decomposition or partitioning of the LP would extend the model beyond the current dense limit toward the 10^6-task graphs we can already generate structurally. Adaptive heuristic budgets: scaling the GA's population and generations with n, or seeding it with the greedy solution, should recover the accuracy lost at large sizes. Richer failure semantics: replication across nodes, checkpointing, or re-scheduling inside the model would go beyond the retry-once model used here. Online adaptation and field deployment: re-scheduling as workloads, links, and node availability change during an event, and validating assumptions on a real federated fog testbed.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback. This research is supported by the National Science Foundation (NSF).

Appendix

This appendix shows that the formulation of Section III-D means what it should, and how to write it for a standard MILP solver. A solution is feasible if every task is placed on exactly one permitted node, at most one l_{ia} per task is non-zero, and all tasks complete by $t_{\max}$.

Lemma 1. If $P_{ia} = 0$ then $e_{ia} = 0$.

Proof. Immediate from (4) and $e_{ia} \in \{0, 1\}$. □

Lemma 2. Every task is assigned to exactly one node.

Proof. By (3), $\sum_a e_{ia} = 1$ with $e_{ia} \in \{0, 1\}$. □

Lemma 3. For each task T_i, at most one l_{ia} is non-zero, and $l_{ia} > 0$ implies $e_{ia} = 1$.

Proof. $l_{ia} > 0 \Rightarrow e_{ia} \geq l_{ia} > 0 \Rightarrow e_{ia} = 1$ by (5), and Lemma 2 leaves no other node with $e_{ia} = 1$. □

It is possible that an instance of the problem has several feasible solutions with different values M_{ia}: the model does not force start times to be minimal, it only requires that no task starts before its inputs have arrived.

Lemma 4. In any feasible solution, the start time M_{ia} respects data arrival: T_i cannot start on f_a before every predecessor has completed and its output has reached f_a.

Proof. Induction on the distance of T_i from a source. A source task has $\text{in}_i = \varnothing$, so $M_{ia} = 0$ by (7) and the claim is vacuous. For the induction step, suppose the claim holds for every task whose distance from a source is less than k, and let T_i have distance k. Every $T_j \in \text{in}_i$ completes at y_j on its assigned node $f_{b^\star}$; the data then needs $c_{ji}^{b^\star a} = \sum_b e_{jb} c_{ji}^{ba}$ to reach f_a, so T_i cannot start on f_a before $y_j + \sum_b e_{jb} c_{ji}^{ba}$. Taking the maximum over $j \in \text{in}_i$, (6) gives

$$M_{ia} \;\geq\; \max_{j \in \text{in}_i} \Big(y_j + \sum_{f_b \in F} e_{jb}\, c_{ji}^{ba} \Big). \tag{14}$$

Thus every M_{ia} in a feasible solution is a valid start time. Setting M_{ia} to exactly this bound recovers the

earliest-start schedule as a special case, but nothing in the formulation requires it: larger values that still satisfy (9) describe equally valid, if later, schedules. □

Lemma 5. If T_i is assigned to f_a (i.e., $e_{ia} = 1$),

1) $M_{ia} \le y_i$ and
2) $y_i = M_{ia} + t_{ia} l_{ia}$ is the completion time of T_i,

where M_{ia} satisfies the data-arrival bound of Lemma 4.

Proof. Follows from (8) and Lemma 4: T_i starts on its assigned node no earlier than its inputs arrive, and runs for $t_{ia} l_{ia}$ time units from that start. □

Lemma 6. All tasks T_i complete by $y_i \le t_{\max}$.

Proof. By (9), after Lemma 5 identifies y_i as the completion time of T_i. □

Theorem 1. Feasible solutions of (2)–(10) and valid schedules are in one-to-one correspondence: every feasible solution describes a valid schedule in which each task runs on one permitted node, starts only after its inputs arrive, and finishes by $t_{\max}$; and every valid schedule yields a feasible solution of equal accuracy. In particular, the optimal value of (2) is the maximum achievable average weighted accuracy.

Proof. For the first direction, Lemmas 1–6 show that a feasible (e, l, M, y) is a valid schedule, and the objective (2) counts exactly its achieved accuracy. For the converse, take any valid schedule and set $e_{ia} = 1$ on the node that runs T_i, l_{ia} to the fraction executed, M_{ia} to the realized start time, and y_i to the realized completion time. Because the schedule is valid—one permitted node per task, inputs received before start, deadline met—the assignment satisfies (3)–(10), and its objective value equals the schedule's accuracy. Since the two sets correspond, the MILP optimum equals the best achievable accuracy. □

For an MILP solver, some constraints need rewriting. First, let us denote $U = t_{\max} + \max_{j,i,b,a}\{c_{ji}^{ba}\}$, and upper bound the M_{ia} variables as

$$M_{ia} \le U, \qquad \forall f_a,\ T_i. \tag{15}$$

Then, to deal with the fact that (8) contains the products $e_{ia} M_{ia}$, which are quadratic, we introduce variables $z_{ia} = e_{ia} M_{ia}$ and replace (8) with

$$y_i = \sum_{f_a \in F} z_{ia} + \sum_{f_a \in F} t_{ia} l_{ia}, \qquad \forall T_i, \tag{16}$$

together with standard constraints (z-constraints),

$$0 \le z_{ia} \le e_{ia}\, U, \qquad \forall f_a,\ T_i, \tag{17}$$

$$M_{ia} - (1 - e_{ia})\, U \le z_{ia} \le M_{ia}, \qquad \forall f_a,\ T_i. \tag{18}$$

These constraints force $z_{ia} = M_{ia}$ when $e_{ia} = 1$, and $z_{ia} = 0$ otherwise.

Theorem 2. The linearized formulation admits exactly the same feasible (e, l) pairs as (2)–(10), and attains the same optimal value of (2).

Proof. Let (e, l, M, y) be a feasible solution for the original formulation. Observe that if any M_{ia} value in this solution does not satisfy (15), it can be set to $M_{ia} = U$, because every $y_j \le t_{\max}$ by (9) and every $c_{ji}^{ba} \le \max_{j,i,b,a}\{c_{ji}^{ba}\}$ by definition. Hence, each lower bound in (6) is at most U, and this does not change the objective value or any other variable. This clamping is only ever needed on non-placed nodes, though: for the node $f_{a^\star}$ that actually runs T_i ($e_{ia^\star} = 1$), feasibility already gives $M_{ia^\star} \le y_i \le t_{\max} < U$ via (8) and (9)—otherwise (9) would be violated—so no placed-node value is ever moved. We also record that, in any feasible solution of the original formulation, $M_{ia} \ge 0$ for every i and a: $M_{ia} = 0$ for source tasks by (7), and for every other task (6) yields $M_{ia} \ge y_j + \sum_{f_b \in F} e_{jb} c_{ji}^{ba} \ge 0$ for each predecessor T_j, since $y_j \ge 0$ by induction over the DAG (a source completes at $y_i = t_{ia^\star} l_{ia^\star} \ge 0$, and (8) then gives $y_i \ge 0$ for the remaining tasks).

Then, if $e_{ia} = 1$, we have that (17) gives $0 \le z_{ia} \le U$ and (18) gives $M_{ia} \le z_{ia} \le M_{ia}$ (the U term vanishes). The squeeze in (18) gives $z_{ia} = M_{ia} = e_{ia} M_{ia}$, and (17) holds since $0 \le M_{ia}$ as noted above and $M_{ia} \le U$ by (15). If otherwise $e_{ia} = 0$, we get that (17) gives $0 \le z_{ia} \le 0$ and (18) gives $M_{ia} - U \le z_{ia} \le 0$. Since $M_{ia} \le U$, $M_{ia} - U \le 0$, and $0 \le M_{ia}$ as noted above. Hence, $z_{ia} = 0 = e_{ia} M_{ia}$. Therefore $z_{ia} = e_{ia} M_{ia}$ in all cases. Setting z accordingly, (e, l, M, z, y) is feasible for the linearized formulation: (15) holds after the adjustment above, (17)–(18) hold by the two cases above, and (16) holds because $l_{ia} = 0$ whenever $e_{ia} = 0$ by (5), so

$$\sum_{f_a \in F} z_{ia} + \sum_{f_a \in F} t_{ia} l_{ia} = \sum_{f_a \in F} e_{ia}\big(M_{ia} + t_{ia} l_{ia}\big) = y_i,$$

where the last equality is (8). The value of (2) is unchanged, and the argument applies in particular to an optimal (e, l, M, y) of (2)–(10), so the optimal value of the linearized formulation is at least that of (2)–(10).

Conversely, let (e, l, M, z, y) be any feasible solution of the linearized formulation. If $e_{ia} = 1$, (18) gives $M_{ia} \le z_{ia} \le M_{ia}$; if $e_{ia} = 0$, (17) gives $z_{ia} = 0$. Hence $z_{ia} = e_{ia} M_{ia}$ in all cases, and, using again $l_{ia} = 0$ when $e_{ia} = 0$, (16) reduces to

$$y_i = \sum_{f_a \in F} e_{ia}\big(M_{ia} + t_{ia} l_{ia}\big),$$

which is exactly (8). Since (3)–(7), (9), and (10) are identical in the two formulations, (e, l, M, y) is feasible for (2)–(10) with the same value of (2). The linearized formulation and (2)–(10) therefore admit exactly the same feasible (e, l) pairs, attain the same optimal value of (2), and their optimal (e, l) pairs coincide. □

References

[1] M. Rehman, S. Ali, and F. Hussain, "Fog computing-based task scheduling for latency-sensitive IoT applications," IEEE Internet of Things Journal, vol. 5, no. 4, pp. 286–298, 2018.

[2] W. Cai, Y. Zhang, and M. Chen, "A survey on edge and fog computing for IoT: Challenges and future directions," ACM Computing Surveys, vol. 51, no. 6, pp. 1–36, 2019.

[3] R. F. Hussain and M. A. Salehi, "Resource allocation of Industry 4.0 micro-service applications across serverless fog federation," Future Generation Computer Systems, vol. 154, pp. 479–490, 2024.

[4] C.-H. Hong and B. Varghese, "Resource management in fog/edge computing: A survey on provisioning, infrastructure, and algorithms," ACM Computing Surveys, vol. 52, no. 5, pp. 1–37, 2019.

[5] G. Gupta, A. Gupta, M. A. Salehi, A. Fernández Anta, S. Bhowmick, J. Aguilar, and M. Kourouma, "Towards efficient allocation of tasks in DAG-based workflows across federated fog systems," in 2025 IEEE Cloud Summit, pp. 124–129, IEEE, 2025.

[6] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," IEEE Transactions on Parallel and Distributed Systems, vol. 13, no. 3, pp. 260–274, 2002.

[7] G. Li, H. Tan, L. Liu, H. Zhou, S. Jiang, Z. Han, X.-Y. Li, and G. Chen, "DAG scheduling in mobile edge computing," ACM Transactions on Sensor Networks, 2023.

[8] A. K. Sharma and J. Kunkel, "GrapheonRL: A graph neural network and reinforcement learning framework for constraint and data-aware workflow mapping and scheduling in heterogeneous HPC systems," in Proc. IEEE Computer Software and Applications Conference (COMPSAC), 2025. arXiv:2506.00260.

[9] E. Deelman, K. Vahi, G. Juve, M. Rynge, S. Callaghan, P. J. Maechling, R. Mayani, W. Chen, R. Ferreira da Silva, M. Livny, and K. Wenger, "Pegasus, a workflow management system for science automation," Future Generation Computer Systems, vol. 46, pp. 17–35, 2015.

[10] R. Tanaka, G. Papadimitriou, et al., "Automating edge-to-cloud workflows for science: Traversing the edge-to-cloud continuum with Pegasus," in Proc. 22nd IEEE/ACM Int. Symp. on Cluster, Cloud and Internet Computing (CCGrid), IEEE, 2022.

[11] J. Köster and S. Rahmann, "Snakemake—a scalable bioinformatics workflow engine," Bioinformatics, vol. 28, no. 19, pp. 2520–2522, 2012.

[12] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, "Nextflow enables reproducible computational workflows," Nature Biotechnology, vol. 35, no. 4, pp. 316–319, 2017.

[13] I. Colonnelli, B. Cantalupo, I. Merelli, and M. Aldinucci, "StreamFlow: Cross-breeding cloud with HPC," IEEE Transactions on Emerging Topics in Computing, vol. 9, no. 4, pp. 1723–1737, 2021.

[14] D. D. Sánchez-Gallegos, D. Di Luccio, S. Kosta, J. L. Gonzalez-Compean, and R. Montella, "An efficient pattern-based approach for workflow supporting large-scale science: The DagOnStar experience," Future Generation Computer Systems, vol. 122, pp. 187–203, 2021.

[15] S. Perrotta, C. G. De Vita, G. Mellone, M. E. Santimaria, M. Torquati, J. García Blas, and R. Montella, "Streaming I/O for scientific workflow engine acceleration," Future Generation Computer Systems, vol. 175, 2025.

[16] S. G. Ahmad, C. S. Liew, M. M. Rafique, E. U. Munir, and S. U. Khan, "Data-intensive workflow optimization based on application task graph partitioning in heterogeneous computing systems," in 2014 IEEE Fourth International Conference on Big Data and Cloud Computing, pp. 129–136, IEEE, 2014.

[17] X. Xu, J. Yan, and D. Wu, "Task scheduling and resource allocation in fog computing systems: A survey," IEEE Internet of Things Journal, vol. 8, no. 6, pp. 4407–4420, 2021.

[18] J. Vergara, J. F. Botero, and L. Fletscher, "A comprehensive survey on resource allocation strategies in fog/cloud environments," Sensors, vol. 23, no. 9, p. 4413, 2023.

[19] C. Mouradian, D. Naboulsi, S. Yangui, R. H. Glitho, M. J. Morrow, and P. A. Polakos, "A comprehensive survey on fog computing: State-of-the-art and research challenges," IEEE Communications Surveys & Tutorials, vol. 20, no. 1, pp. 416–464, 2018.

[20] E. Baccarelli, M. Scarpiniti, and A. Momenzadeh, "EcoMobiFog—design and dynamic optimization of a 5G mobile-fog-cloud multi-tier ecosystem for the real-time distributed execution of stream applications," IEEE Access, vol. 7, pp. 55565–55608, 2019.

[21] M. Chiou, G. T. Epsimos, G. Nikolaou, P. Pappas, G. Petousakis, S. Mühl, and R. Stolkin, "Robot-assisted nuclear disaster response: Report and insights from a field exercise," in IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4545–4552, 2022.

[22] M. Gao, L. Wan, R. Shen, Y. Gao, J. Wang, Y. Li, and B. Vucetic, "SparkLink: A short-range wireless communication protocol with ultra-low latency and ultra-high reliability," The Innovation, vol. 4, p. 100386, 2023.

[23] A. Kabanov and V. Kramar, "Marine internet of things platforms for interoperability of marine robotic agents: An overview of concepts and architectures," Journal of Marine Science and Engineering, vol. 10, p. 1279, 2022.

[24] F. Faticanti, F. De Pellegrini, D. Siracusa, D. Santoro, and S. Cretti, "Throughput-aware partitioning and placement of applications in fog computing," IEEE Transactions on Network and Service Management, vol. 17, pp. 2436–2450, 2020.

[25] F. Faticanti, F. De Pellegrini, D. Siracusa, D. Santoro, and S. Cretti, "Load-balancing optimization framework for heterogeneous fog nodes," in 2020 IEEE International Conference on Communications Workshops (ICC Workshops), pp. 1–6, IEEE, 2020.

[26] G. Russo, D. Ferrarelli, D. Pasquali, V. Cardellini, and F. Lo Presti, "QoS-aware offloading policies for serverless functions in the cloud-to-edge continuum," Future Generation Computer Systems, vol. 156, pp. 1–15, 2024.

[27] G. Perin, M. Berno, T. Erseghe, and M. Rossi, "Towards sustainable edge computing through renewable energy resources and online, distributed and predictive scheduling," IEEE Transactions on Network and Service Management, vol. 19, pp. 306–321, 2022.

[28] M. Min, L. Xiao, Y. Chen, P. Cheng, D. Wu, and W. Zhuang, "Learning-based computation offloading for IoT devices with energy harvesting," IEEE Transactions on Vehicular Technology, vol. 68, no. 2, pp. 1930–1941, 2019.

[29] J. Wang, J. Hu, G. Min, W. Zhan, Q. Ni, and N. Georgalas, "Computation offloading in multi-access edge computing using a deep sequential model based on reinforcement learning," IEEE Communications Magazine, vol. 57, no. 5, pp. 64–69, 2019.

[30] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," IEEE Communications Surveys & Tutorials, vol. 19, no. 4, pp. 2322–2358, 2017.

[31] J. Gentry, C. Denninnart, and M. Salehi, "Robust dynamic resource allocation via probabilistic task pruning in heterogeneous computing systems," in Proceedings of IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 375–384, 2019.

[32] M. A. Salehi, J. Smith, A. A. Maciejewski, H. J. Siegel, E. K. Chong, J. Apodaca, L. D. Briceno, T. Renner, V. Shestak, J. Ladd, et al., "Stochastic-based robust dynamic resource allocation for independent tasks in a heterogeneous computing system," Journal of Parallel and Distributed Computing, vol. 97, pp. 96–111, 2016.

[33] J. K. Lenstra, D. B. Shmoys, and É. Tardos, "Approximation algorithms for scheduling unrelated parallel machines," Mathematical Programming, vol. 46, pp. 259–271, 1990.

[34] P. Raghavan and C. D. Thompson, "Randomized rounding: A technique for provably good algorithms and algorithmic proofs," Combinatorica, vol. 7, no. 4, pp. 365–374, 1987.

[35] G. Gupta, "Data-aware scheduling of approximate DAG workflows in federated fog systems: Simulator and experiments." https://github.com/gayanigupta/data-aware-scheduling, 2026.